IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Received 4 January 2026, accepted 22 January 2026, date of publication 3 February 2026, date of current version 19 February 2026.

*Digital Object Identifier 10.1109/ACCESS.2026.3660949*

RESEARCH ARTICLE

# Case Study: PQC Migration for Android Strongbox Keymaster on Samsung Galaxy Mobile Devices

**SUNGMIN LEE[1], HYUNGCHUL JUNG[2], NOHIK HEO[3], AND TED "TAEKYOUNG" KWON[1]**
[1]Department of Computer Science and Engineering, Seoul National University, Seoul 08826, South Korea
[2]Security Center, SK Telecom, Seoul 04539, South Korea
[3]Mobile eXperience (MX) Business, Samsung Electronics, Suwon 16677, South Korea

Corresponding author: Ted "Taekyoung" Kwon (tkkwon@snu.ac.kr)

This work was supported in part by the Institute of Information and Communications Technology Planning and Evaluation (IITP) through the Artificial Intelligence Graduate School Program (Seoul National University) under Grant RS-2021-II211343, in part by the Next-Generation Cloud-Native Cellular Network Leadership Program under Grant IITP-2025-RS-2024-00418784, in part by the Information Technology Research Center (ITRC) Support Program grant funded by Korean Government [Ministry of Science and ICT (MSIT)] under Grant IITP-2025-2021-0-02048, and in part by the National Research Foundation of Korea (NRF) grant funded by MSIT under Grant NRF-2022R1A2C2011221 and Grant RS-2023-00220985.

**ABSTRACT** Advances in quantum computers and their algorithms are significantly changing modern cyber-security systems. Post-Quantum Cryptography (PQC) has been standardized to protect modern security systems from large-scale and reliable quantum computing. However, most PQC algorithms require processing tens to hundreds of times larger key lengths than classical cryptographic algorithms. For example, the RSA-4096 algorithm has a private key of 512 bytes, and the NIST P-521 curve-based ECDSA algorithm has a private key of approximately 66 bytes. However, the FIPS 204 standard PQC digital signature algorithm, ML-DSA-87, has a private key length of 4,896 bytes. This large key length is a heavy burden for Strongbox Keymaster, one of the fundamental modules of the Android security system. This is because the Secure Execution Environment (SEE) in which Android Strongbox Keymaster operates is severely resource-constrained. Therefore, we propose a novel scheme by analyzing the characteristics of modern Android security environments and designing PQC scenarios. Our scheme preserves the hardware-backed security level of Android devices and introduces a new key slotting mechanism to improve the PQC ML-DSA performance of eSE. Through this, in an experimental environment based on real Android devices, our scheme speeds up the PQC ML-DSA-65 signing performance of Strongbox Keymaster by 597 ms (1,826 → 1,229 ms, about 33%) compared to the drop-in replacement scheme.

**INDEX TERMS** Android, embedded secure element (eSE), galaxy devices, hardware-backed, keymaster, keystore, mobile security, module-lattice-based digital signature algorithm (ML-DSA), post quantum cryptography (PQC), smartphone, strongbox, samsung electronics.

## I. INTRODUCTION

Advances in quantum computing are significantly impacting modern cybersecurity. As large-scale and reliable quantum computers are developed, modern cybersecurity systems may become vulnerable to Shor's algorithm [1] and Grover's algorithm [2], [3]. A qubit in quantum computing is the smallest unit of computation, like a bit in classical computers. In theory, approximately 4K-qubit-scale quantum computing can crack the RSA-2048 cryptographic algorithm [4]. In addition, 4K-qubit-scale quantum computing can also crack the ciphers of the National Institute of Standards and Technology (NIST) P-521 elliptic curve cryptography (ECC) algorithm [5]. Many quantum computing experts predict that 4K-qubit-scale quantum computing will be achievable around 2030 [6], [7]. In 2022, the United States Congress legislated a transition to quantum-resistant cryptography within 5 years of standard publication [8]. Then in 2024, the NIST standardized new cryptographic algorithms called Post-Quantum Cryptography (PQC) that would remain secure even after large-scale and reliable quantum computing is realized [3], [7], [9], [10].

The associate editor coordinating the review of this manuscript and approving it for publication was Mostafa M. Fouda.

Most PQC algorithms have larger key lengths than the algorithms they replace [3]. For example, the private key of the Federal Information Processing Standardization (FIPS) 204, Module-Lattice-based Digital Signature Standard-87 (ML-DSA-87) digital signature algorithm, one of the PQC standards, is 4,896 bytes [11]. This length is approximately ten times larger than the 512-byte private key of the RSA-4096 algorithm, which is widely used for similar purposes in modern cybersecurity systems. It is also about 74 times larger than the approximately 66-byte private key of the Elliptic Curve Digital Signature Algorithm (ECDSA), which is based on the NIST P-521 curve. Although the purpose may be somewhat different, it is 153 times larger than the 32-byte secret key of the Advanced Encryption Standard (AES)-256 algorithm, which has a security strength equivalent to Level 5 of ML-DSA-87 in the new security strength categories that include both post-quantum and classical cryptographic algorithms [7].

Integrating the large keys of PQC algorithms as drop-in replacements into modern cybersecurity systems is challenging [3]. Schwabe et al. showed that when PQC was applied as a drop-in replacement scheme to the standard TLS 1.3 security protocol, which is easy to replace algorithms, in an experimental environment modeling the real world, the payload size increased by about 50% (2,711 → 4,056 bytes) to up to 13,022% (2,711 → 355,737 bytes) depending on the combination of algorithms [12]. Similarly, implementing PQC Attestation in Android Strongbox Keymaster as a drop-in replacement scheme would result in severe performance degradation. This is because modern Android Strongbox Keymaster operates on security hardware that offers a high-security level but severely limited resources [13], [14].

Android Keymaster is a module dedicated to cryptographic operations on Android devices along with the Keystore [13]. It has a similar purpose as Apple's Secure Enclave [15] along with CryptoKit [16]. Early hardware-backed Keymaster ran in a Trusted Execution Environment (TEE) within the Application Processor (AP), isolated from the AP's general execution environment, called the Rich Execution Environment (REE). However, for recent Android mobile devices, the Keymaster runs on an embedded Secure Element (eSE) that is physically separated from the AP. We call the execution environment provided by the eSE the Secure Execution Environment (SEE). And to distinguish the Keymaster operating in SEE from the TEE-based Keymaster, we call it the Strongbox Keymaster [14].

The overall security level of an Android device is generally defined by the security level of the TEE provided by the AP. This is because the AP's TEE can generate a device-wide Root of Trust (RoT) using the unique values of peripherals such as the device's display and flash memory. The security level of TEE is at least the Evaluation Assurance Level (EAL) 2+ based on the Protection Profile (PP) of the Common Criteria (CC) [17]. APs are fundamentally designed for interoperability with other peripherals and fast performance. Thus, the TEE provided by the AP may be vulnerable to side-channel attacks [18], [19], and it is generally difficult to achieve a high level of security from the PP perspective. However, eSEs are designed independently and in a closed manner to achieve a high-security level as the top priority rather than performance. Thus, eSEs generally have countermeasures against hardware attacks, including side-channel attacks such as response time analysis and energy consumption measurement [20], [21], [22]. Therefore, the security level of SEE is at least EAL4+ based on CC PP [23]. However, eSE is fundamentally isolated from other peripherals, making it may be vulnerable to chip-swap attacks [24], [25]. As a result, SEE is hard to support device-wide integrity.

Modern Android devices selectively perform SEE-based functions requiring higher security than TEE. The mobile Driver's License (mDL) [26] and digital car keys [27] are widely known SEE-based functions. Attempting to perform all computations of a device in a highly secure SEE is still overly idealistic. Due to the fundamentally different design goals, the performance and resources of eSE are significantly restricted compared to AP. For example, the specifications of the S3FV9RRP model, one of Samsung Electronics' high-end eSEs, are Core 70MHz, RAM 53KB, and Flash 2MB [28]. This execution environment is severely resource-constrained compared to AP with several GHz of Core, several GB of RAM, and hundreds of GB or a few TB of flash storage. Considering the eSE's OS and hardware acceleration solutions, the resources available to applications operating in the SEE are further limited. Furthermore, PQC's key length and computational amount, which are much larger than those of classical cryptographic algorithms, make it even more insufficient. Nevertheless, the Android Keymaster is required to support at least 8,192 keys [29]. As a result, the storage utilization of eSE is chosen constrainedly. Typically, lock screen-related functions on Android devices such as Weaver [30] are prioritized to utilize SEE [31].

Many studies have demonstrated that ML-DSA, a PQC digital signature standard based on Dilithium [32], is a suitable quantum-resistant digital signature algorithm in resource-constrained environments among other PQC algorithms [33], [34], [35], [36]. However, more efforts are necessary to design a practical and applicable scheme. Our study introduces a new scheme for the Android Strongbox Keymaster to efficiently and securely support PQC ML-DSA digital signatures. This scheme incorporates a novel key slotting mechanism that actively utilizes eSE's restricted resources. Then, it speeds up the ML-DSA-65's performance by 597 ms (1,824 → 1,227 ms, approximately 33%) in an experimental environment. As a result, Android applications requiring PQC migration and eSE's high-security level can perform attestations faster.

The contributions of this study are as follows. First, this study summarizes the cybersecurity threats from quantum computing in an easy-to-understand manner. This background knowledge will motivate cybersecurity experts who

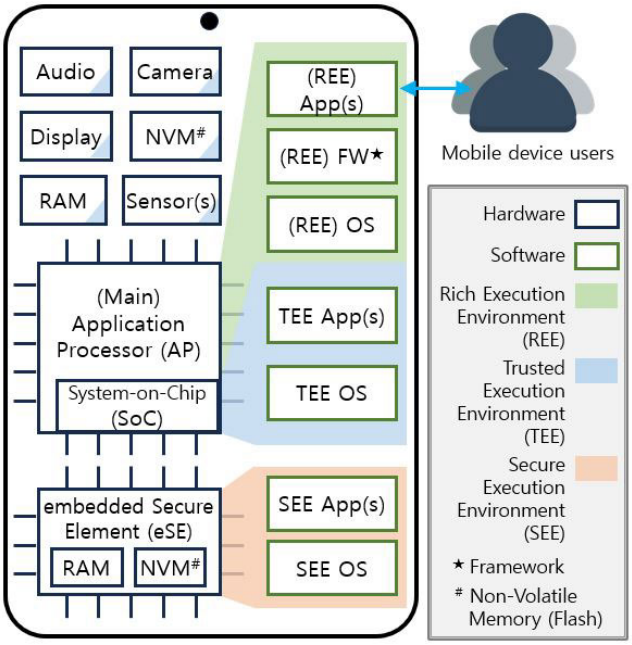


**FIGURE 1.** Illustrates three execution environments of modern Android devices. REE is a general execution environment. TEE is an AP manufacturer-guaranteed trusted environment. SEE is eSE manufacturer-guaranteed, more secure than the TEE. See the details in Section II-A.

are hesitant to undertake PQC migration. Second, this study thoroughly details and analyzes the pain problem when Android Strongbox Keymaster supports PQC ML-DSA in a drop-in replacement scheme. This serves as a referable case study for mobile system experts who will face the challenges of PQC migration with constrained resources. Finally, we introduce a novel eSE key-slotting mechanism faster than the drop-in replacement scheme. While preserving the device security level, this mechanism mitigates the problems caused by the large keys of the PQC algorithms and the eSE's low performance, which has been acceptable for classical cryptographic algorithms. This mechanism and overall scheme can provide insights into addressing various issues due to the PQC drop-in replacement scheme in diverse cases. To our best knowledge, this is the first study to efficiently and securely support the final version of the PQC ML-DSA standards on Android devices. Consequently, after success in device-wide quality assuarance and security assessment processes, the proposed scheme has been integrated into future commercial devices.

The rest of the paper is organized as follows: Section II covers Android security execution environments and security threats from quantum computing as backgrounds. Section III defines the challenges of this study in detail. Section IV briefly introduces several studies about PQC optimization and its applications in constrained environments. Section V presents our design and proposed scheme. Section VI compares and evaluates our scheme with the drop-in replacement scheme. Section VII shares additional topics that are beyond the scope of this study but worth mentioning from the mobile system security perspective. Finally, Section VIII concludes our study.

## II. BACKGROUNDS

### A. ANDROID SECURITY EXECUTION ENVIRONMENTS

Figure 1 shows the three execution environments that modern Android devices have. In terms of performance and resources, REE > TEE $\gg$ SEE. On the other hand, from a security level perspective, the order is REE $\ll$ TEE < SEE.

#### 1) TRUSTED EXECUTION ENVIRONMENT (TEE) ON AP

Most modern Android and security-critical devices have the AP, which offers hardware-backed security features. These features divide AP's execution environment into the general REE and secure TEE. Thus, even if attackers gain top-level privileges of the REE, direct access to the TEE is generally restricted.

TEE is a secure execution environment guaranteed by the AP manufacturer. Typically, TEE is implemented as a System-on-Chip (SoC), which separates the executions at the register level inside the AP. In the case of TrustZone, a well-known TEE of ARM architecture, it is accessible only via Secure Monitor when the AP raises Exception Level 3 (EL3) [37]. Depending on the AP manufacturer, TEE has various names, such as Samsung's TEEGris, Qualcomm's QSEE and QTEE, and MediaTek's MobiCore and Kinibi.

APs are designed not only for fast performance but also for compatibility with peripherals. Thus, it is possible to derive the device-wide RoT by collecting the invariable unique values of peripheral hardware. In this way, TEE can guarantee the device-wide integrity. Even if the SEE described in the following section has a higher security level than the TEE, the TEE's security level determines the device's overall security level.

A software execution module that runs in the TEE is called a Trusted Application (TA). Basically, TEE is read-only and does not have its own storage. Thus, when a TA needs to store data, the TA encrypts the data with a symmetric encryption key that is not exposed outside the TEE and stores the encrypted data in REE's storage. Therefore, the performance of TAs on the same AP is generally a bit slower than REE's apps.

#### 2) SECURE EXECUTION ENVIRONMENT (SEE) ON eSE

TEE is physically separated from REE at the register level inside the AP, but peripherals such as RAM and Flash outside the AP are not physically separated. Instead, it logically separates the execution environment through encryption and access control mechanisms. Therefore, attackers generally cannot see the plaintext of data that TAs stored in the REE. However, if the attackers gain top-level privileges in the REE, they can access encrypted data stored there. If attackers arbitrarily compromise or delete the encrypted critical data, the device may experience a panic state.

Therefore, when a more secure execution environment than TEE is required, the device offers SEE that physically

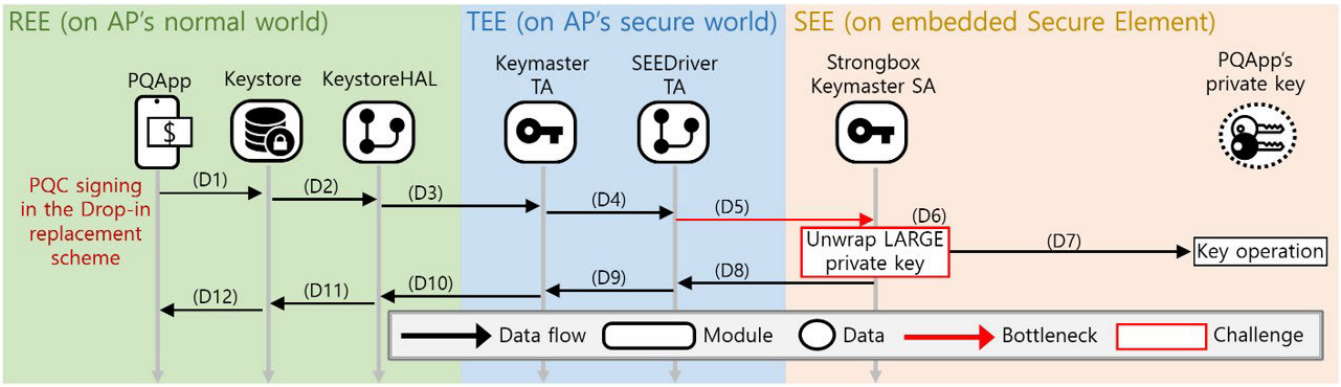


**FIGURE 2.** Illustrates a challenge when Android Keymaster serves PQC signing as a drop-in replacement. (D5) Transmitting a large wrapped PQC private key and (D6) Unwrapping it in the SEE heavily burdens the eSE of low performance and resource-constrained. See each step's detail in the Section III.

isolates all operations from the AP, including peripherals such as RAM and Non-Volatile Memory (NVM). We call this execution environment SEE, and in this study, we refer to a software execution module running in SEE as a Secure Application (SA)[1]. The eSE is the widely used name for hardware chips that provide SEE. eSE has different names depending on the mobile device manufacturer, such as Samsung's Knox Vault [22], [31], Google's Titan-M [38], and Apple's T2 [39].

The operations performed in TEE and SEE are based on classical cryptographic algorithms. For example, Secure Boot typically combines the Secure Hash Algorithm (SHA) with the RSA (or ECC) algorithms [40]. Lock screen-related features are mainly implemented using the Hash-based Message Authentication Code (HMAC) algorithm [41]. File-Based Encryption (FBE) for storage protection uses HMAC and AES algorithms [42].

### B. SECURITY THREATS FROM QUANTUM COMPUTING

#### 1) OVERVIEW OF QUANTUM COMPUTING

Quantum computing is a technology that utilizes the quantum mechanical properties of particles, such as photons, electrons, and ions, to carry out parallel computations [43], [44], [45], [46]. As notable differences from classical computing, this paper briefly refers to the superposition and entanglement of qubits, as well as the reversibility of quantum computing logic gates.

First, a bit, the smallest unit of classical computing, has a definite state of 0 or 1. On the other hand, a qubit, the smallest unit of quantum computing, can have a state with probabilities of 0 and 1 simultaneously. Until a qubit is observed, it is in a state of probabilities of 0 and 1. Once observed, the qubit's probabilities are collapsed, and its state is determined as the observed value of 0 or 1. This property is called **Superposition** [44], [45].

Second, qubits can be placed in a state of **Entanglement** under the control of quantum logic gates. Once one of the entangled qubits is observed, its probability state collapses, immediately affecting the probability states of the remaining qubits that had entangled with the observed qubit. This effect occurs spontaneously and simultaneously, independent of time and distance. Therefore, the computation required to transmit or synchronize the results of certain conditions in classical computing can be omitted in quantum computing. Applying this point, for difficult problems where all possible cases must be explored to find a unique or optimal value in a huge problem space, the entanglement property can be utilized to perform high-level parallel computations [43].

Finally, similar to classical computing, which consists of AND or OR gates, etc., quantum computing also consists of standard logic gates, such as Hadamard and Toffoli [44], [46]. However, unlike classical computing, all quantum logic gates are guaranteed to be reversible. **Reversibility** is the property of specifying the input value by the result value. In classical computing, the only gate that guarantees reversibility is the NOT gate. In contrast, other gates, such as AND and OR, have cases where the input value cannot be specified as the result value[2]. On the other hand, operators used in quantum computing can be represented as matrices, and every operator matrix has its inverse matrix. Therefore, the operations performed in quantum computing can inversely compute the input values using the output values [45], [46].

Quantum computing is applicable across various fields, such as machine learning, natural sciences, finance, and optimization [47]. However, its most notable application is in cryptanalysis [3], [7], [43]. The following section details how quantum computing can crack or weaken classical cryptographic algorithms.

#### 2) QUANTUM CRYPTANALYSIS

The two representative quantum computing algorithms are Shor's [1] and Grover's [2].

First, Shor's prime number factorization algorithm leverages the periods of repeating identical values in a

[1]The names of software modules running in SEE may vary depending on the development environment. For example, if SEE is based on JavaCard OS, software modules on JavaCard OS are commonly called JavaCard Applets.

[2]If the output of the OR gate is 1, the input value was one of [0, 1], [1, 1], or [1, 0], but no one can specify which one. Similarly, if the output of the AND gate is 0, the input value was one of [1, 0], [0, 0], or [1, 0], but we cannot specify which one.

**TABLE 1. Shows TEE-SEE transmission rates and AES-256-CBC performances in the experimental setup. Classical cryptographic algorithms take only a few ms, but PQC ML-DSA private keys take several hundred ms. Large PQC private keys can seriously degrade the device user experience.**

| Data Sizes | 32 B (AES-256) | 66 B (ECDSA-521) | 512 B (RSA-4096) | 1 KB | 2 KB | 2,560 B (ML-DSA-44) | 3 KB | 4,032 B (ML-DSA-65) | 4,896 B (ML-DSA-87) | 5 KB |
|---|---|---|---|---|---|---|---|---|---|---|
| TEE-SEE one-way transmission (ms) | 5 | 8 | 62 | 117 | 193 | 221 | 248 | 306 | 348 | 364 |
| AES-256-CBC encryption on eSE (ms) | 8 | 13 | 105 | 179 | 241 | 264 | 278 | 301 | 349 | 369 |
| AES-256-CBC decryption on eSE (ms) | 6 | 9 | 81 | 143 | 161 | 174 | 215 | 296 | 319 | 343 |

finite field. In quantum computing, periodic values can be computed in polynomial time via the Quantum Fourier Transform (QFT). Therefore, quantum computing can solve the factorization problem, which requires exponential time in classical computing, in polynomial time (exponential speed up) [3], [7], [43].

Second, the Grover algorithm is a search algorithm that iterates the inverse transformation on the problem space's average value to find the operation's initial input value with a specific result value [2], [44]. This approach enables searches of unstructured databases (DBs) by square-root time in quantum computing (quadratic speed-up) [3], [7], [44].

The security strength of classical cryptography represents the number of possible cases in the problem space as an exponential bit [48]. For example, the AES-128 algorithm, which has 128-bit security strength, uses one of about $2^{128}$ possible keys. Also, the 112-bit security strength of the RSA-2048 algorithm means that the number of cases to choose two 1024-bit prime numbers is greater than $2^{112}$ but less than $2^{113}$. Therefore, once large-scale quantum computing with reliable operations is realized, RSA cryptography, based on the difficulty of the prime factorization problem, is no longer secure via Shor's algorithm [3], [7], [43]. This is because the prime factorization problem will no longer pose the same level of exponential difficulty. Similarly, ECC and Diffie-Hellman (DH) algorithms, which are based on the difficulty of the discrete logarithm problem, are also insecure [3], [5], [7]. On the other hand, Grover's algorithm can reduce the bit-wise security strength of symmetric-key ciphers and one-way ciphers (hash algorithms) by about half [3], [7], [49].

NIST released standard PQC algorithms to ensure cybersecurity systems remain secure even after quantum computing advances [7], [8], [9], [10], [11]. Many quantum computing experts predict that quantum computers threatening classical cryptography will be developed around 2030 [7], [33].

## III. CHALLENGE DEFINITION

Android Keymaster is categorized into Keymaster TA operating in TEE and Strongbox Keymaster SA operating in SEE [14]. Our design primarily addresses the PQC Strongbox Keymaster SA in SEE. Figure 2 shows a typical problem when Android Strongbox Keymaster supports PQC in a drop-in replacement scheme.

The detailed steps for Android apps to perform PQC ML-DSA signing on the device are as follows. **(D1)** A security-sensitive Android app running in the REE initiates the signature generation by calling the Keystore Application Programming Interface (API) in the Android framework. This call includes the message to be signed and the alias of the private key, not the private key itself. **(D2)** The Keystore in the REE receives the app's request, replaces the alias with the wrapped (i.e., encrypted) private key, and forwards the app's request to the Keystore's Hardware Abstraction Layer (KeystoreHAL). **(D3)** The KeystoreHAL forwards the Keystore's request to the Keymaster TA in the TEE. **(D4)** The Keymaster TA sends this request to the SEEDriver TA. **(D5)** The SEEDriver TA forwards the request to the PQCKeymaster SA in the SEE through the channel between the TEE and SEE. **(D6)** The PQCKeymaster SA already has the SEE wrapper key. Hence, it unwraps (i.e., decrypts) the app's wrapped private key in the SEE. **(D7)** The PQCKeymaster SA generates a PQC digital signature in the SEE using the app's plaintext private key. **(D8-11)** The signature generated by the PQCKeymaster SA is returned to the Keystore in the REE via the SEEDriver TA, the Keymaster TA, and the KeystoreHAL. **(D12)** The Keystore API returns the generated signature to the Android app.

The problem occurs in steps D5 and D6. First, the PQC key, which is tens to hundreds of times larger than the classical cryptographic algorithms, causes unacceptable delays at step D5. Table 1 shows the transmission rates between TEE and SEE measured on a Samsung Galaxy S23 FE device. While the 512-byte private key of the RSA-4096 algorithm and the 66-byte private key of the ECDSA-521 algorithm for the same digital signature purpose are acceptable transmission rates, several kilobytes of the PQC ML-DSA private key adversely affect the device user experience. Specifically, in our experimental environment, the time required to transmit the private key of ECDSA-521 algorithm takes 8 ms. On the other hand, ML-DSA-87 takes 348 ms.

The second problem is the encryption/decryption of the app's key that occurs in step D6. Table 1 also shows the AES-256-CBC encryption/decryption performance measured in an experimental environment. Similarly, encrypting/decrypting large keys of PQC on low-performance SEEs negatively impacts the device user experience. For example, in an experimental environment, the time required to decrypt a 66-byte private key of the ECDSA-521 algorithm inside SEE is only 9 ms. In contrast, the time to decrypt a 4,896-byte private key of ML-DSA-87 inside SEE is 319 ms. In conclusion, adopting PQC ML-DSA in the Android Strongbox Keymaster as a drop-in replacement scheme can

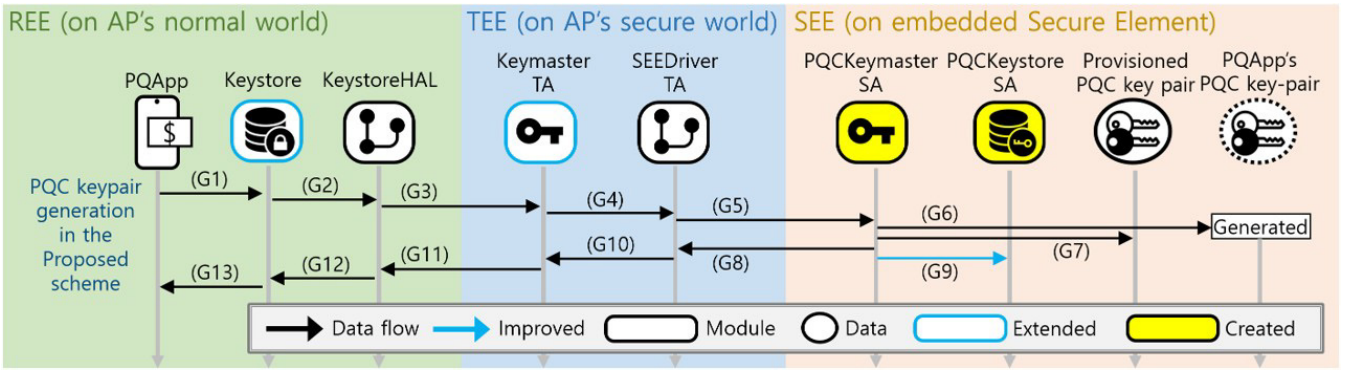


**FIGURE 3.** Illustrates the proposed scheme's key-pair generation phase. Our scheme has nine components, as explained in Section V-B. PQCKeymaster SA and SEEKeystore SA are implemented from scratch. Keystore TA and Keymaster TA are extended. An additional but not time-consuming G9 step streamlines later PQC ML-DSA signing. See each step's details in Section V-C1.

result in significant delays that are unforeseen from classical cryptographic algorithms and may cause seriously negative impact on the user experience.

## IV. RELATED WORKS

This section introduces studies on PQC optimization and its application in resource-constrained environments. Then, we compare them with our study.

Roy and Kalita [34] studied the computational complexity required for standardized PQC algorithms to operate on resource-constrained devices. Their study proves that lattice-based algorithms such as Dilithium [32] perform best in resource-constrained environments. However, we show that even lattice-based ML-DSA is challenging for Android strongbox Keymaster under the drop-in replacement scheme.

Señor et al. [50] analyzed NTRU, one of the well-known PQC algorithms, and studied a scheme to operate efficiently on resource-constrained IoT devices. They also demonstrated efficient utilization of the PQC algorithm in IoT devices by leveraging cryptographic hardware accelerators. However, our study prioritizes final PQC standard algorithms, such as ML-DSA, from a practical interoperability perspective.

Ebrahimi et al. [51] studied an efficient architecture to support quantum-resistant cryptography using Ring-LWE in resource-constrained IoT environments. Their proposed IoT-based quantum-resistant cryptographic support is fast and lightweight but focuses on processor-level optimization. In contrast, our study focuses on the PQC optimization of Android Strongbox Keymaster, an widely used dedicated cryptographic module on eSE.

Gonzalez et al. [35] studied to verify PQC signatures with 8KB of memory. They compared PQC algorithms through signature verification via streaming in an environment widely used in the automotive field. Similar to [34], their study also shows that Dilithium [32], standardized as ML-DSA [11], is more useful in constrained environments than Rainbow, GeMSS, and SPHINCS+ algorithms. However, we should also consider key-pair generation and signing for the real-world Android mobile device experience.

Emura et al. [52] proposed a caching system to reduce the communication overhead on the Internet due to the large key of PQC. Their study proposes an encrypted cache system that considers the catch-22 situation where the cache server violates end-to-end encryption to observe the content. Although our study focuses on mobile devices rather than web environments, we present a new key-slotting mechanism that serves a similar purpose to the proposed caching system.

Pratiwi et al. [53] implemented Dilithium and Kyber from CRYSTALS in an Intel SGX environment and analyzed memory and performance. SGX, the TEE of the Intel processor, is one of the widely used Key Management Systems (KMSs). Their study addresses supporting PQC in a TEE suitable for a server environment. In contrast, our study emphasizes optimizing PQC in SEE of mobile devices, where resources are significantly more constrained than TEE.

## V. PROPOSED SCHEME

Our design divides the execution phase of the PQC ML-DSA algorithm into a key-generation phase and a key-utilization phase. Next, we add a key-slotting mechanism to the key-generation phase. A new SA, SEEKeystore, plays a primary role in this mechanism. Later in the key-utilization phase, our scheme skips the transmission of the encrypted ML-DSA private key to the SEE and the ML-DSA private key decryption inside the SEE. Therefore, our scheme speeds up the execution time of the Android app's ML-DSA signing.

Typically, the eSE of existing mobile devices does not store Android AppKeys (the App's private or secret keys) in the eSE's storage. This is because the storage space of the eSE is significantly limited due to the space used by the SEE OS, SA, and the SA's master key. On the contrary, this study actively utilizes the eSE's non-volatile storage.

Also, caching generally refers to the use of temporary memory to improve performance. On the contrary, the slotting introduced in this study permanently stores AppKeys until the App or Keystore explicitly requests the PQCKeymaster SA to delete them. Therefore, slotting is functionally different from the temporary data implied by typical caching mechanisms.

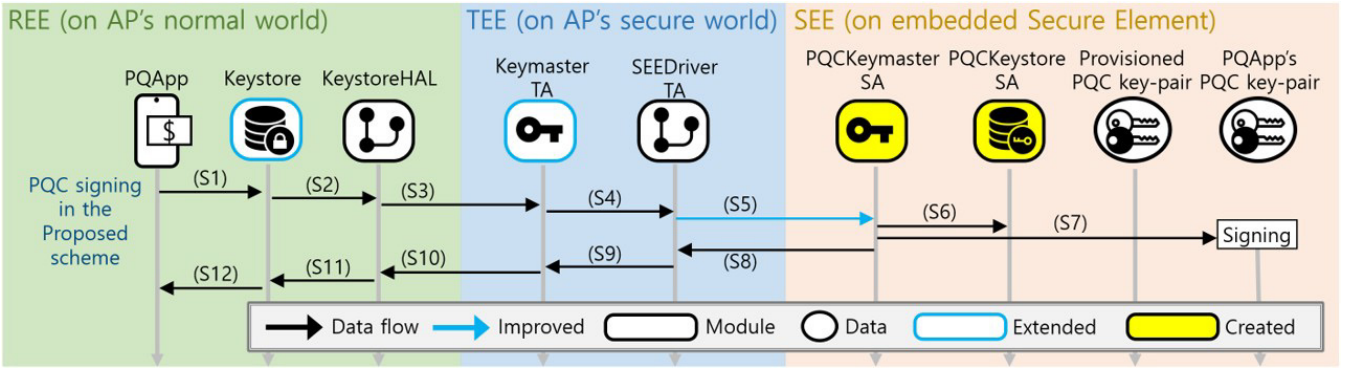


**FIGURE 4.** Illustrates the proposed scheme's PQC signing. Thanks to the slotted private key in SEEKeystore, Keystore can skip the transmission of the encrypted PQApp's PQC private key, and PQCKeymaster can omit the decryption of the PQApp's private key. These speed up the PQApp's PQC signing. See each step's details in Section V-C2.

### A. DESIGN GOALS

In this section, we briefly address the objectives pursued by our scheme.

#### 1) PRESERVING HARDWARE SECURITY LEVEL

The performance and security level of the device are often in a trade-off. But we shouldn't sacrifice security for better performance. In our scheme, hardware-backed keys are not exposed outside the hardware. Any symmetric secret and asymmetric private keys generated in SEE are only visible within SEE, and any symmetric secret and asymmetric private keys utilized in SEE are never exposed in plaintext outside SEE.

#### 2) SUSTAINABILITY

Intuitively, the high-security level of SEE and the performance improvement through the key-slotting mechanism inside SEE proposed in this study are attractive to security-sensitive apps that want to utilize PQC on devices. However, considering the Android device-wide efficiency, the constrained storage of SEE should be allocated to the most frequently called keys. Therefore, we prepare a framework to apply various preemptive scheduling mechanisms, such as Least Recently Used (LRU), from the design phase.

#### 3) TRANSPARENCY

In our design, SEEKeystore, which leverages the resource-constrained storage of SEE, collaborates with the Android Keystore in REE. Thus, apps utilizing SEEKeystore and available resources in SEE are transparently disclosed to Android framework and device users through the Android Keystore. Each Android app can be informed of the availability of SEE resources at runtime, and the device user can reclaim SEE resources allocated to individual apps through the Android framework when necessary.

### B. SCHEME COMPONENTS

As shown in Figure 3, our scheme consists of nine components. First, in REE, a **PQApp** is any app that wants to utilize PQC. Most mobile banking, payments, and e-commerce apps can be one of PQApps. **Keystore** is the module of the Android framework [14]. This provides APIs related to cryptographic operations that Android apps and systems can utilize. Also, Keystore has a DB that stores Android app's wrapped keys encrypted with hardware-backed wrapper keys. Thus, REE apps cannot access the plaintext of the wrapped secret or private key. **KeystoreHAL** is a dedicated module for communication between Keystore in REE and Keymaster TA in TEE[3].

Next, in the TEE, **Keymaster** handles cryptographic operations on the Android system. The existing Keymaster TA does not support PQC, but we extended it. Additionally, Keymaster TA still protects the keys of other apps using Wrapper keys that the AP manufacturer guarantees they are not exposed outside the TEE and derives the RoT using unique values of peripherals. This RoT is used to ensure device-wide hardware integrity. **SEEDriver** TA is dedicated to communication between TEE and SEE. Via SEEDriver TA, Keymaster TA binds to PQCKeymaster SA, making it the only communication channel with SEE.

Finally, in SEE, **PQCKeymaster** SA is responsible for PQC cryptographic operations on the device. Like Keymaster TA, which has Wrapper keys that are not exposed outside the TEE, PQCKeymaster SA has SEEWrapper keys that are not exposed outside the SEE. In addition, PQCKeymaster can access the **Provisioned PQC key-pair**, which was provided by HSM equipment during device manufacturing. **SEEKeystore** is a module that manages the storage of the SEE. Unlike other SAs, SEEKeystore keeps the keys of PQApps inside SEE even when the module is updated. For this purpose, we implemented the Global Platform (GP) Amendment H mechanism [54]. The **Provisioned PQC key-pair** is a unique PQC ML-DSA key-pair for each SEE. Depending on the implementation, ML-DSA-65 key-pair can be selected. **PQApp's PQC key-pair** is generated at the PQApp's request. Generally, the private key of this key-pair is encrypted in the SEE and stored in the REE's Keystore, but through our Key-slotting mechanism, it is selectively stored inside the SEE as well.

[3]Generally, HAL modules provide standard interfaces to a variety of hardware.

### C. PQC DIGITAL SIGNATURE OPERATIONS

PQC digital signature algorithms, including ML-DSA, consist mainly of three APIs: key-pair generation, signing, and signature verification. This section details ML-DSA key-pair generation and signing steps related to the proposed scheme. The remaining signature verification is mentioned in Section VII-A. Afterward, we describe the APIs related to the SEE key management.

#### 1) ML-DSA KEY-PAIR GENERATION

Figure 3 shows the steps when PQApp requests PQC key-pair generation. **(G1)** PQApp calls Keystore's PQC key-pair generation API. At this time, PQApp assigns a new alias of the key-pair to be generated. **(G2-5)** The request of the PQApp starts from the Keystore and is passed through KeystoreHAL, Keymaster TA, and SEEDriver TA to the PQCKeymaster SA. At this time, the Keystore can set the isSEESlotPreferred flag to true. **(G6)** PQCKeymaster generates a PQC key-pair for the PQApp. **(G7)** The public key of the generated key-pair is formed as an X.509 certificate using the private key of the Provisioned PQC Key-pair. **(G8)** According to the isSEESlotPreferred flag value, PQCKeymaster assigns a SEESlotID to the PQApp's private key and encrypts the private key with its SEEWrapKey. Afterward, it returns the SEESlotID, the encrypted PQC private key, and the X.509 certificate, which includes the PQC public key, to the SEEDriver. **(G9)** Finally, the PQCKeymaster stores the PQApp's PQC private key and the SEESlotID in SEEKeystore. **(G10-12)** The results generated by PQCKeymaster are returned to Keystore through SEEDriver, Keymaster TA, and KeystoreHAL. **(G13)** Keystore stores the outputs of PQCKymaster and returns the result of key-pair generation to PQApp.

#### 2) ML-DSA SIGNING

Figure 4 shows the steps when PQApp requests to generate an ML-DSA digital signature using the PQApp's ML-DSA private key, which is generated as described in Section V-C1. **(S1)** PQApp calls Keystore's API with a message to be signed and the alias of the key-pair. **(S2)** Keystore searches the DB using the PQApp's key-pair alias. If PQApp's alias has SEESlotID, Keystore does not send PQApp's encrypted PQC private key but sends the SEESlotID value and the message to be signed. **(S3-5)** Keystore's request is delivered to the PQCKeymaster SA via KeystoreHAL, Keymaster TA, and SEEDriver TA. **(S6)** PQCKeymaster checks the SEESlotID to get PQApp's PQC private key from SEEKeystore. **(S7)** PQCKeymaster generates the PQC signature using PQApp's private key. **(S8-S12)** The generated signature is returned to the PQApp through SEEDriver TA, Keymaster TA, KeystoreHAL, and Keystore.

#### 3) SEE SLOT MANAGING OPERATIONS

Keystore is a module of the Android framework [55]. When the device user removes an app or factory-resets the device, Keystore deletes the keys generated by the app. At this time, the app's keys slotted in SEEKeystore should also be deleted. For this purpose, our scheme implements **unslotSeeKey** API in Keystore. This API also allows the device user to cancel a specific PQApp's SEE slot usage. Conversely, Keystore provides **slotSeeKey** API to slot an unslotted private key back into the SEE. At this time, the X.509 certificate issued by the PQCKeymaster SA and PQApp's wrapped PQC private key are sent to the PQCKeymaster SA so that the key to be reslotted can be verified as issued by the SEE.

In addition, device users and Android apps can check the status of the SEE with Keystore APIs. Before generating a PQC key-pair, PQApps can check whether the SEE key slot is available using the **isSeeSlotAvaiable** API. Also, **getSeeSlotState** API provides the total available space and the currently used space of the SEE storage.

The call stacks for SEE slot managing APIs are from Keystore to the SEEKeystore via KeystoreHAL, Keymaster, SEEDriver, and PQCKeymaster.

**TABLE 2.** Shows the experimental setup's eSE specifications. The eSE OS and eSIM-related specifications are excluded. Regardless of total eSE resources, the maximum available storage is 1 MB, and the RAM is 92 KB. Our study implements PQCKeymaster and SEEKeystore, then optimizes PQC operations in this resource-constrained environment.

| | | | |
|---|---|---|---|
| CPU | Core | SC300 | 160 MHz |
| Memory | Total | NVM - Flash | 3.5 MB |
| | | VM - RAM | 136 KB |
| | Available | NVM - Flash | 1 MB |
| | | VM - RAM | 92 KB |
| Interface | Available | SPI, NFC, UWB | 12.5 - 20 MHz |

## VI. EVALUATIONS

In this section, we evaluate and compare the proposed scheme with the drop-in replacement scheme.

### A. EXPERIMENTAL SETUP

The experimental environment is a Samsung Galaxy S23 FE device, and the message size to be signed is fixed at 2KB. The validity of 2KB message size for the experimental setup is further addressed in VII-H3. The eSE providing SEE on this device is S3NSEN6 [56]. Table 2 shows the detailed specifications of the eSE related to the experiments.

We actually implemented the ML-DSA-44 and ML-DSA-65 algorithms in the experimental environment,and measured their performances. However, in experimental environments, running ML-DSA-87 algorithm is unavailable due to insufficient resources. Thus, the performance of ML-DSA-87 was simulated based on CRYSTALS, the original author of the ML-DSA algorithm, released CPU cycle data [57], private key and signature length of ML-DSA-87, SEE Unwrapping and channel performance measured in Table 1, and the performances of step-by-step ML-DSA-44 and ML-DSA-65. The validity of the ML-DSA-87 simulation is further addressed in VII-I.

### B. ML-DSA KEY-PAIR GENERATION

Figure 5 shows the elapsed times when PQApp requests to generate a PQC ML-DSA key-pair on the Android device. The time required for each security strength was 1,628 ms for ML-DSA-44 and 2,046 ms for ML-DSA-65. The ML-DSA-87 algorithm, simulated based on measured and official factors, takes 2,334 ms. As shown in step G9 in Figure 3, the key-slotting of the PQCKeymaster is performed after the key-pair generation response. Therefore, compared to the drop-in replacement scheme, our scheme does not increase the time for the key-pair generation.

### C. ML-DSA SIGNING

Figures 6, 7, and 8 show the performances of ML-DSA algorithms by security strength. In each figure, the left red-series bars are the elapsed times for steps of the drop-in replacement scheme, and the right blue-series bars are the times taken for steps of the proposed scheme. Each line shows the cumulative time of the below bars. First, Figure 6 presents the performance of ML-DSA-44 in the experimental setup. The proposed scheme reduces the accumulated time by 418 ms, improving performance by about 31% (1,351 → 933 ms). Second, Figure 7 presents the performance of ML-DSA-65 in the experimental setup. The proposed scheme reduces the accumulated time by 597 ms, improving performance by about 33% (1,826 → 1,229 ms). Lastly, Figure 8 presents the performance of ML-DSA-87 simulated based on the published data and the measured data in the experimental setup. The proposed scheme is expected to reduce the accumulated time by 597 ms, improving performance by about 33% (1,826 → 1,229 ms).

### D. EVALUATION ANALYSIS

Table 3 shows the ML-DSA signing performance profile. In the performance profile of each algorithm, the steps where the proposed scheme provides performance improvement are TEE → SEE and the unwrapping PQC private key inside SEE. The performances of other steps are the same. The proposed scheme reduces or eliminates the unavoidable overhead of Android Strongbox Keymaster, which should be handled at the low-performance and resource-constrained SEE. Furthermore, the proposed scheme aims to handle several kilobytes of large ML-DSA private keys efficiently. Thus, the performance improvement of the proposed scheme is not affected by the length of the message to be signed. Similarly, since the signature length of each strength is fixed, the length of the ML-DSA signature generated at the request of the PQApp does not affect the performance improvement of the proposed scheme. In other words, the proposed scheme achieves deterministic performance improvement by efficiently processing the PQApps's ML-DSA private key, regardless of the length of the message to be signed and the length of the generated signature. For example, when PQApp generates ML-DSA-65 digital signatures in an experimental environment, the proposed scheme achieves a performance improvement of 597 ms for both 1KB and 10KB to-be-signed

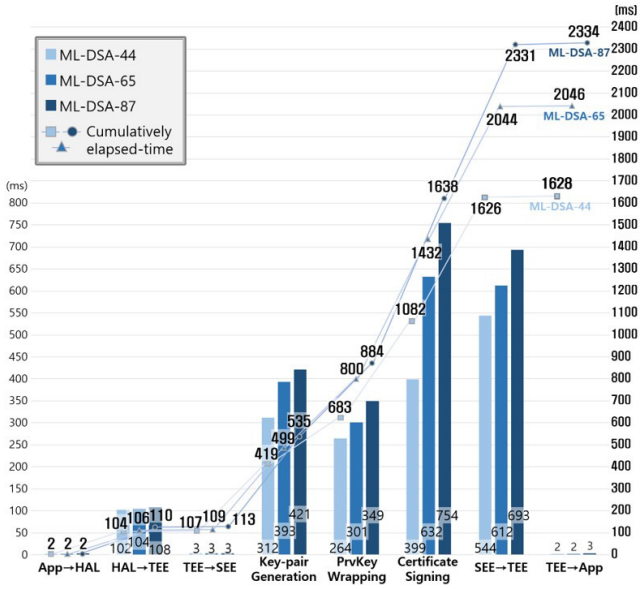


**FIGURE 5.** Illustrates ML-DSA key-pair generation performances. Rectangulars show elapsed times of each step, and lines show accumulated times for each algorithm's steps. As described in Figure 3 and Section V-C1, our scheme doesn't affect key-pair generation performances.

messages. The performance benefis under different traffic patterns is further addressed in VII-H3.

## VII. DISCUSSIONS

### A. ML-DSA SIGNATURE VERIFICATION

Verification of ML-DSA signatures, which is not detailed in this study, is the operation with public keys and independent of the private key. Therefore, the security level of the execution environment required for signature verification is relatively lower than that of operations that generate or utilize private keys. From the Android device's perspective, signature verification can be conducted by the Keystore, which is protected by Android system permission in the REE, or a Keymaster TA operating in the AP's TEE.

### B. AVAILABLE OPENSOURCES

The SEE development environment may vary depending on each device. However, other researchers who want to extend this study further in an environment similar to ours can implement the content of this paper utilizing opensources like JavaCardKeymaster, released in [58], and ML-DSA from Crystals, released in [59]. Also, most PQC algorithm implementations can be found in the Open Quantum Safe (OQS) [60].

### C. SECURITY ASSESSMENT

New modules must not introduce new vulnerabilities to the device. Basically, the proposed scheme collaborates with the default Android security features. Even the Android OS of REE, which has the lowest security level, has various security mechanisms applied [61]. For example, the Keystore located in the Android framework is protected by the Application

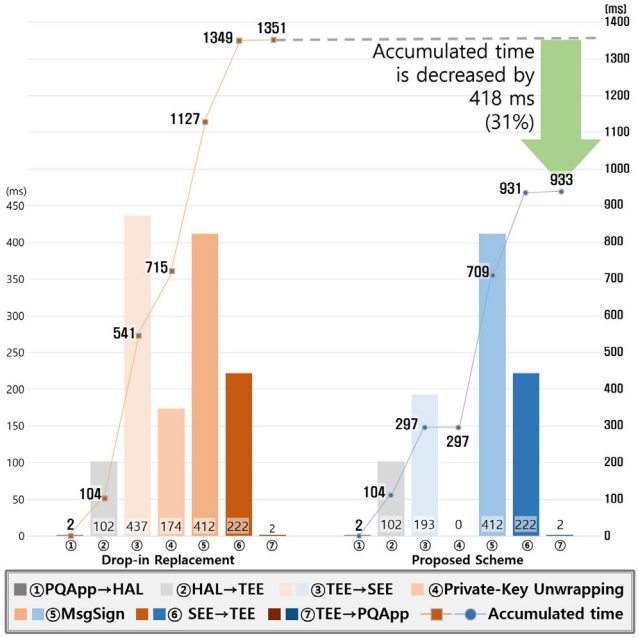


**FIGURE 6.** Illustrates the ML-DSA-44 signing performance step-by-step. The bar graph shows the elapsed time of each step, and the line graph shows the accumulated time to sign. Compared to the drop-in replacement, our scheme reduces the signing time by 31% (1,351 → 933 ms).

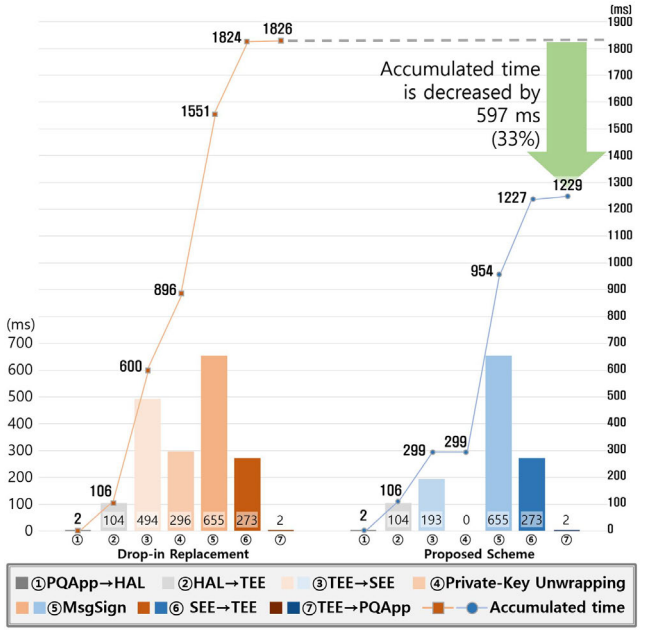


**FIGURE 7.** Illustrates the ML-DSA-65 signing performance step-by-step. The bar graph shows the elapsed time of each step, and the line graph shows the accumulated time to sign. Compared to the drop-in replacement, our scheme reduces the signing time by 33% (1,826 → 1,229 ms).

sandbox and system permissions. Therefore, if the device is not rooted, any 3rd party apps cannot access Keystore's DB. In addition, KeystoreHAL binds to the Keymaster TA at the time of the device's first out-of-box boot, making it the only access to the Keymaster. Afterward, Keymaster TA ignores requests from any other modules except Keystore's requests through KeystoreHAL. The TEE has various security mechanisms (e.g., AP manufacturer - OEM dual signing [40]) that ensure the integrity and access control of the TEE even if the Android OS in the REE is rooted. Similarly, SEEDriver TA binds to SEE's PQCKeymaster at the device manufacturing. Therefore, PQCKeymaster ignores requests from any other modules except the Keymaster TA's requests via SEEDriver TA. The SEE incorporates various security mechanisms (e.g., Fault Detection, etc.) that allow the eSE manufacturer to ensure the integrity and access control of the SEE even if the TEE is compromised. Finally, Android devices adopting the proposed scheme have passed the manufacturer's demanding security assessments before commercialization.

### D. PQC KEM OPERATIONS

PQC algorithms are categorized into digital signature algorithms and Key Encapsulation Mechanisms (KEMs). KEM algorithms mainly have three APIs: key-pair generation, encapsulation, and decapsulation. Unlike the PQC digital signature algorithms, which have APIs similar to the classical digital signature algorithms, the KEM algorithm has a slightly different usage scenario. The encapsulation API uses the public key to generate two outputs: an encapsulated (i.e., encrypted) and plaintext symmetric secret key. The decapsulation API uses the private key to decrypt (i.e., decapsulate) another's encrypted symmetric key to recover the plaintext symmetric secret key, generated by the other. From the perspective of the Android device, the generated or decapsulated symmetric secret key should not be exposed to the REE as plaintext, so wrapping and unwrapping in the TEE or SEE are required. Optimizing PQC KEM APIs to be faster than the drop-in replacement scheme of Android Keymaster for this requirement is our future work.

### E. SEE SLOT SCHEDULING

Considering SEE's insufficient storage space and large PQC keys, SEE's key slots are rare resources. In order to keep the user's device experience optimal, several mechanisms can be applied in addition to the LRU mentioned above. The optimization focuses on ensuring that the private keys used by the device user are being slotted in the SEE as often as possible. Additionally, we need to be able to manage key slots so that malicious or not frequently used apps do not all monopolize them. In our first implementation, the newly generated private key can always be slotted immediately, and the least-used key is unslotted when no more slots are left. It also limits the number of slots available to individual apps. In the future, machine learning algorithms may be applied to make a scheduling policy based on the prediction of each app's key-slot demand for the device user.

### F. PQC OPERATIONS IN TEE

As described in Section V-A1, our design aims to preserve the high-security level of SEE. However, when SEE is

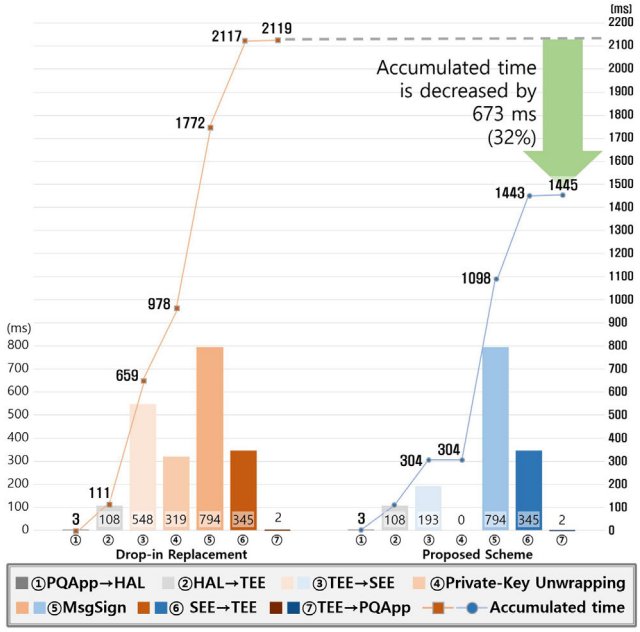


**FIGURE 8.** Illustrates the ML-DSA-87 signing performance step-by-step. The bar graph shows the elapsed time of each step, and the line graph shows the accumulated time to sign. Compared to the drop-in replacement, our scheme is expected to reduce the signing time by 32% (2,119 → 1,445 ms).

unavailable, or SEE's low performance and insufficient resources are unacceptable, TEE-based PQC operation may be a necessary fallback. This is because, at least, it is way more secure than handling private keys in REE or maintaining classical cryptographic algorithms that are not quantum-resistant. As described in Section II-A1, TEE's performance is slightly slower than REE's but much faster than SEE's. In our experimental setup, each ML-DSA operation performed in the TEE takes just tens of milliseconds. The specific signing times are 21 ms for ML-DSA-44, 32 ms for ML-DSA-65, and 38 ms for ML-DSA-87. Also, private key unwrapping times are equally 6 ms. Times of other steps for TEE-based PQC implementation, such as transmission from PQApp to TEE, are the same.

### G. HANDLING MESSAGES LARGER THAN SEE MEMORY

Similar to the implementation of classical digital signature algorithms, PQC ML-DSA can sign large messages with limited memory through the Init - Update - Finalize steps. For messages larger than the SEE's memory, the Update step is repeatedly performed for split messages to generate the signature for total message.

### H. EXPERIMENTAL MISCELLANEOUS

#### 1) POWER CONSUMPTION OF eSE

This study did not address energy consumption. While unrelated to the topic of the paper, from a mobile device perspective, the energy consumption of eSEs is largely insignificant compared to APs and other peripherals.

#### 2) PERFORMANCE IN MULTI-THREADING

The experimental performances were measured on a single thread. The TEE-to-SEE communication discussed in the paper is typically atomically synchronized using a single polling-based channel at the current state of the art. As the paper notes, this is because the performance gap between APs and eSEs is very large. Therefore, the results for multitasking and repeated single-tasking will likely be similar.

#### 3) PERFORMANCE BENEFITS UNDER DIFFERENT TRAFFIC PATTERNS

As mentioned in the VI-D, the performance improvement of the proposed scheme is related to the large key processing required by the PQC algorithm, which achieves a fixed-time gain. Therefore, the percentage of performance improvement achieved by the proposed scheme in a single digital signature processing may vary depending on the size of the message to be signed. In the real-world implementation, the message to be signed will largely be a hash value of the original message. If the SHA-384 algorithm is used, the message size to be signed will be only 48 bytes. In this case, the percentage contribution from the fixed-time performance improvement can be significantly higher. If the app requests a Certificate Signing Request (CSR) for PQC attestation, the message size will be approximately 2-3 KB, even considering the large public key size of ML-DSA. In this case, the percentage contribution from the fixed-time performance improvement will be slightly lower than the level presented in the paper.

Modern Android devices have a combined execution environment that links REE, TEE, and SEE, and the time required for an digital signature generated by an app located in the REE has many variables (e.g., performance of AP and eSE, key wrapping/unwrapping performance, channel performance between each execution environment, message size, eSE buffer, Maximum Transmission Unit (MTU) and Maximum Segment Size (MSS) provided by SEE OS, etc.). Although the performance improvement presented by the experimental setup was set to normalize various real-world scenarios, we tried presenting the absolute ms values and the relative percentage together for easier comparison.

### I. ML-DSA-87 SIMULATION

We confirmed that the increase in the execution time of the computation step (412 ms → 655 ms, approximately 59%) between the actually implemented ML-DSA-44 and ML-DSA-65 on eSE is almost similar to the increase (333,013 → 529,106, approximately 58%) in the amount of computation (CPU cycles). Therefore, we trust that the increase in the amount of computation of ML-DSA-87 compared to ML-DSA-65 (529,106 → 642,192 in cycle, approximately 21%) will lead to an increase in the pure signing execution time (655 → 794 in ms, approximately 21%). As mentioned in VI-A, The performance measurement data of Crystals, which serves as the basis for the simulation, is cited as [57] in the paper. Except for the ML-DSA signing step,

**TABLE 3. Shows the ML-DSA signing performance profile. The proposed scheme achieves deterministic performance improvement by efficiently processing the PQApps's ML-DSA private key. Performances of steps irrelevant to the private key are the same. (time unit: ms).**

| Category | Scheme | App→TEE | TEE→SEE | PrKeyUwrap | MsgSign | SEE→TEE | TEE→App | Total |
|---|---|---|---|---|---|---|---|---|
| ML-DSA-44 | Drop-in replacement | 104 | 437 | 174 | 412 | 222 | 2 | 1,351 |
| | Proposed scheme | 104 | 193 | 0 | 412 | 222 | 2 | 933 |
| ML-DSA-65 | Drop-in replacement | 106 | 494 | 296 | 655 | 273 | 2 | 1,826 |
| | Proposed scheme | 106 | 193 | 0 | 655 | 273 | 2 | 1,229 |
| ML-DSA-87 | Drop-in replacement | 111 | 548 | 319 | 794 | 345 | 2 | 2,119 |
| | Proposed scheme | 111 | 193 | 0 | 794 | 345 | 2 | 1,445 |

other data transmission steps also follow a linear increase in transmission time with data size. We present the simulated times for each step in Table 3. Therefore, we can assure that the performance of ML-DSA-87 when actually implemented will be similar to the simulation results presented in the paper, under the same eSE performance and channel speed.

## VIII. CONCLUSION

This study summarizes the security execution environments of modern Android devices and the security threats of quantum computing. Next, we propose a novel design to efficiently and securely support PQC in the Android Strongbox Keymaster. Our scheme actively utilizes the resource-constrained and low-performance eSE efficiently, securely, and transparently while preserving the high-security level of the SEE provided by the eSE. In the experimental setup, our scheme improves the performance of the ML-DSA algorithm, a NIST final PQC digital signature standard, by up to 33%. Our proposed scheme has been submitted for an international patent and has been commercialized.

## REFERENCES


[1] P. W. Shor, "Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer," *SIAM Rev.*, vol. 41, no. 2, pp. 303–332, Jan. 1999.

[2] L. K. Grover, "A fast quantum mechanical algorithm for database search," in *Proc. 28th Annu. ACM Symp. Theory Comput. (STOC)*, 1996, pp. 212–219.

[3] L. Chen, S. Jordan, Y.-K. Liu, D. Moody, R. Peralta, R. Perlner, and D. Smith-Tone, "Report on post-quantum cryptography," U.S. Dept. Commerce, NISTIR, Nat. Inst. Standards Technol., Gaithersburg, MD, USA, Tech. Rep. 8105, 2016. [Online]. Available: https://nvlpubs.nist.gov/nistpubs/ir/2016/nist.ir.8105.pdf

[4] S. Beauregard, "Circuit for shor's algorithm using 2n+3 qubits," 2002, *arXiv:quant-ph/0205095*.

[5] T. Häner, S. Jaques, M. Naehrig, M. Roetteler, and M. Soeken, "Improved quantum circuits for elliptic curve discrete logarithms," in Proc. Int. Conf. Post-Quantum Cryptogr. Cham, Switzerland: Springer, 2020, pp. 425–444.

[6] M. Mosca and M. Piani. (2024). *2024 Quantum Threat Timeline Report*. Global Risk Insitute. [Online]. Available: https://globalriskinstitute.org/publication/2024-quantum-threat-timeline-report/

[7] Dustin Moody. (2022). *The Beginning of the End: The First NIST PQC Standards*. NIST. [Online]. Available: https://csrc.nist.gov/csrc/media/Presentations/2022/the-beginning-of-the-end-the-first-nist-pqc-standa/images-media/pkc2022-march2022-moody.pdf

[8] U.S. Congress. (2022). *H.R.7535-Quantum Computing Cybersecurity Preparedness Act*. [Online]. Available: https://www.congress.gov/bill/117th-congress/house-bill/7535/text

[9] NIST. (2024). *NIST To Standardize Encryption Algorithms That Can Resist Attack By Quantum Computers*. [Online]. Available: https://www.nist.gov/news-events/news/2023/08/nist-standardize-encryption-algorithms-can-resist-attack-quantum-computers

[10] U.S. Department of Commerce. (2024). *Announcing Issuance of Federal Information Processing Standards (FIPS) FIPS 203. Module-Lattice-Based Key-Encapsulation Mechanism Standard, FIPS 204, Module-Lattice-Based Digital Signature Standard, and FIPS 205, Stateless Hash-Based Digital Signature Standard*. https://www.federalregister.gov/documents/2024/08/14/2024-17956/announcing-issuance-of-federal-information-processing-standardsfips-fips-203-module-lattice-based

[11] NIST. (2024). *FIPS 204 Module-Lattice-Based Digital Signature Standard*. [Online]. Available: https://csrc.nist.gov/pubs/fips/204/final

[12] P. Schwabe, D. Stebila, and T. Wiggers, "Post-quantum TLS without handshake signatures," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Oct. 2020, pp. 1461–1480.

[13] Google. (2024). *Hardware-Backed Keystore*. [Online]. Available: https://source.android.com/docs/security/features/keystore

[14] Google. (2024). *Hardware Security Module*. [Online]. Available: https://developer.android.com/privacy-and-security/keystoreHardwareSecurityModule

[15] Apple. (2024). *Secure Enclave*. [Online]. Available: https://support.apple.com/en-us/guide/security/sec59b0b31ff/web

[16] Apple. (2024). *Apple CryptoKit*. [Online]. Available: https://developer.apple.com/documentation/cryptokit

[17] Global Platform. (2020). *TEE Protection Profile V1.3*. [Online]. Available: https://globalplatform.org/specs-library/tee-protection-profile-v1-3

[18] X. Lou, T. Zhang, J. Jiang, and Y. Zhang, "A survey of microarchitectural side-channel vulnerabilities, attacks, and defenses in cryptography," *ACM Comput. Surv.*, vol. 54, no. 6, pp. 1–37, Jul. 2022.

[19] J. Wichelmann, A. Pätschke, L. Wilke, and T. Eisenbarth, "Cipherfix: Mitigating ciphertext side-channel attacks in software," in Proc. 32nd USENIX Secur. Symp. (USENIX Security), 2023, pp. 6789–6806.

[20] T. Schläpfer and A. Rüst, "Security on IoT devices with secure elements," in Proc. Embedded World Conf., Feb. 2019, pp. 1–8.

[21] L. Zimmerli, T. Schläpfer, and A. Rüst, "Securing the IoT: Introducing an evaluation platform for secure elements," in Proc. Konferenz Internet Things-vom Sensor bis zur Cloud, Stuttgart, Germany, 2019.

[22] Samsung. (2025). *Knox Vault-Protection From Attacks*. [Online]. Available: https://docs.samsungknox.com/admin/fundamentals/whitepaper/samsung-knox-for-android/core-platform-security/knox-vault

[23] Euro Smart. (2014). *Security IC Platform Protection Profile With Augmentation Packages*. [Online]. Available: https://www.commoncriteriaportal.org/files/ppfiles/pp0084b_pdf.pdf

[24] M. Roland, J. Langer, and J. Scharinger, "Practical attack scenarios on secure element-enabled mobile devices," in *Proc. 4th Int. Workshop Near Field Commun.*, Mar. 2012, pp. 19–24.

[25] M. Kim, J. Suh, and H. Kwon, "A study of the emerging trends in SIM swapping crime and effective countermeasures," in *Proc. IEEE/ACIS 7th Int. Conf. Big Data, Cloud Comput., Data Sci. (BCD)*, Aug. 2022, pp. 240–245.

[26] Google. (2024). *Google Wallet*. [Online]. Available: https://wallet.google/digitalid/

[27] Google. (2024). *Digital Car Key*. [Online]. Available: https://www.android.com/digital-car-key

[28] Samsung. (2024). *S3FV9RRP Specification*. [Online]. Available: https://semiconductor.samsung.com/security-solution/ese-esim/part-number/s3fv9rrp-prime

[29] Google. (2024). *Android 15 Compatibility Definition-9.11. Keys and Credentials*. [Online]. Available: https://source.android.com/docs/compatibility/15/android-15-cdd911_keys_and_credentials

[30] Google. (2024). *Weaver*. [Online]. Available: https://android.googlesource.com/platform/hardware/interfaces/+/refs/heads/master/weaver/1.0/IWeaver.hal
[31] Samsung. (2024). *Knox Vault Features-Weaver*. [Online]. Available: https://docs.samsungknox.com/admin/fundamentals/whitepaper/samsung-knox-for-android/core-platform-security/knox-vault/weaver
[32] L. Ducas, E. Kiltz, T. Lepoint, V. Lyubashevsky, P. Schwabe, G. Seiler, and D. Stehlé, "Crystals-dilithium: A lattice-based digital signature scheme," IACR Transactions Cryptograph. Hardw. Embedded Syst., pp. 238–268, 2018.
[33] M. Kumar, "Post-quantum cryptography Algorithm's standardization and performance analysis," *Array*, vol. 15, Sep. 2022, Art. no. 100242.
[34] K. S. Roy and H. K. Kalita, "A survey on post-quantum cryptography for constrained devices," Int. J. Appl. Eng. Res., vol. 14, no. 11, pp. 2608–2615, 2019.
[35] R. Gonzalez, A. Hülsing, M. J. Kannwischer, J. Krämer, T. Lange, M. Stöttinger, E. Waitz, T. Wiggers, and B. Y. Yang, "Verifying post-quantum signatures in 8 kB of RAM," in Proc. Int. Conf. Post-Quantum Cryptogr. Cham, Switzerland: Springer, 2021, pp. 215–233.
[36] M. Raavi, P. Chandramouli, S. Wuthier, X. Zhou, and S.-Y. Chang, "Performance characterization of post-quantum digital certificates," in *Proc. Int. Conf. Comput. Commun. Netw. (ICCCN)*, Jul. 2021, pp. 1–9.
[37] ARM. (2015). *ARM Cortex-A Series Programmer's Guide for ARMv8-A*. [Online]. Available: https://developer.arm.com/documentation
[38] Google. (2018). *Titan M Makes Pixel 3 Our Most Secure Phone Yet*. [Online]. Available: https://blog.google/products/pixel/titan-m-makes-pixel-3-our-most-secure-phone-yet
[39] Apple. (2018). *Apple T2 Security Chip*. [Online]. Available: https://www.apple.com/kr/mac/docs/Apple_T2_Security_Chip_Overview.pdf
[40] Qualcomm. (2019). *Secure Boot and Image Authentication*. [Online]. Available: https://www.qualcomm.com/content/dam/qcomm-martech/dm-assets/documents/secure-boot-and-image-authentication-version_final.pdf
[41] Google. (2025). *Authentication*. [Online]. Available: https://source.android.com/docs/security/features/authentication
[42] Qualcomm. (2025). *File Based Encryption*. [Online]. Available: https://www.qualcomm.com/content/dam/qcomm-martech/dm-assets/documents/file_based_encryption_enhancements_final_06.10.2019.pdf
[43] V. Bhatia and K. R. Ramkumar, "An efficient quantum computing technique for cracking RSA using shor's algorithm," in *Proc. IEEE 5th Int. Conf. Comput. Commun. Autom. (ICCCA)*, Oct. 2020, pp. 89–94.
[44] O. Morsch, *Quantum Bits and Quantum Secrets: How Quantum Physics is Revolutionizing Codes and Computers*. Hoboken, NJ, USA: Wiley, 2008.
[45] Roger A. Grimes, *Cryptography Apocalypse: Preparing for the Day When Quantum Computing Breaks Today's Crypto*. Hoboken, NJ, USA: Wiley, 2019.
[46] J. L. Hevia, G. Peterssen, C. Ebert, and M. Piattini, "Quantum computing," *IEEE Softw.*, vol. 38, no. 5, pp. 7–15, Sep. 2021.
[47] The Quantum Insider. (2024). *5 Crucial Quantum Computing Applications & Examples*. [Online]. Available: https://thequantuminsider.com/2023/05/24/quantum-computing-applications
[48] E. Barker, *Part 1 Rev. 5 Recommendation for Key Management: Part 1—General*, document NIST SP 800-57, NIST, 2020.
[49] C. Zalka, "Grover's quantum searching algorithm is optimal," *Phys. Rev. A*, vol. 60, no. 4, p. 2746, 1999.
[50] J. Señor, J. Portilla, and G. Mujica, "Analysis of the NTRU post-quantum cryptographic scheme in constrained IoT edge devices," *IEEE Internet Things J.*, vol. 9, no. 19, pp. 18778–18790, Oct. 2022.
[51] S. Ebrahimi, S. Bayat-Sarmadi, and H. Mosanaei-Boorani, "Post-quantum cryptoprocessors optimized for edge and resource-constrained devices in IoT," *IEEE Internet Things J.*, vol. 6, no. 3, pp. 5500–5507, Jun. 2019.
[52] K. Emura, S. Moriai, T. Nakajima, and M. Yoshimi, "Cache-22: A highly deployable end-to-end encrypted cache system with post-quantum security," *Cryptol. ePrint Arch.*, 2022.
[53] N. Pratiwi, M. R. Firmansyah, and M. F. Ezerman, "Implementing CRYSTALS kyber and dilithium in Intel SGX secure enclaves," in *Proc. IEEE Int. Conf. Cryptogr., Informat., Cybersecurity (ICoCICs)*, Aug. 2023, pp. 77–81.
[54] Global Platform. (2018). *Executable Load File Upgrade-Amendment H*. [Online]. Available: https://globalplatform.org/specs-library/globalplatform-technology-executable-load-file-upgrade-amendment-h-v1-1
[55] Google. (2025). *Android Keystore System*. [Online]. Available: https://developer.android.com/privacy-and-security/keystore
[56] Samsung. (2024). *S3NSEN6 Specification*. [Online]. Available: https://semiconductor.samsung.com/security-solution/nfc/part-number/s3nsen6
[57] Crystals. (2024). *Dilithium Performance Overview*. [Online]. Available: https://pq-crystals.org/dilithium
[58] S. Willden. (2024). *Github-JavaCardKeymaster*. [Online]. Available: https://github.com/divegeek/JavaCardKeymaster
[59] Crystals. (2024). *Github-Dilithium*. [Online]. Available: https://github.com/pq-crystals/dilithium
[60] Open Quantum Safe. (2024). *Github-Open Quantum Safe*. [Online]. Available: https://github.com/open-quantum-safe
[61] Google. (2024). *Secure Android Devices*. [Online]. Available: https://source.android.com/docs/security

**SUNGMIN LEE** received the B.S. degree in computer science and engineering from Inha University, in 2007, and the M.S. degree in computer science from Yonsei University, in 2014. He is currently pursuing the Ph.D. degree in computer science and engineering with Seoul National University (SNU). From 2014 to 2020, he was a Mobile Security Software Engineer with LG Electronics. Since 2022, he has been a Mobile Security Software Engineer with Mobile eXperience (MX) Business, Samsung Electronics. His research interests include cryptography and its applications, such as post quantum cryptography (PQC) migration, mobile system security, network, and hardware security.

**HYUNGCHUL JUNG** received the M.S. degree in cybersecurity from Ajou University, in 2004. From 2021 to November 2025, he was the Head of the Security Engineering Group, Mobile eXperience (MX) Business, Samsung Electronics. Since November 2025, he has been the Head of security operations and detection office with SK Telecom. He is currently a seasoned security expert with over 15 years of experience spanning security architecture design, development, and commercialization for mobile devices and services.

**NOHIK HEO** received the M.S. degree in electronic engineering from Kyungpook National University, Republic of Korea. Since 2005, he has been with Samsung Electronics, where he is currently a Principal Engineer. Since 2015, he has been engaged in security-related research, with a focus on data protection, manufacturing processes, and kernel security. His current research interests include system software security, secure boot architectures, and security enhancement for embedded systems.

**TED "TAEKYOUNG" KWON** received the B.S., M.S., and Ph.D. degrees from the Department of Computer Engineering, Seoul National University (SNU), South Korea, in 1993, 1995, and 2000, respectively. He was a Visiting Professor with Rutgers University, in 2010. Before joining SNU, he was a Postdoctoral Research Associate with the University of California, Los Angeles (UCLA), and City University New York (CUNY). During his graduate program, he was a Visiting Student with the IBM T. J. Watson Research Center and the University of North Texas. He is currently a Professor with the School of Computer Science and Engineering, SNU. His research interests include future internet, content-oriented networking, and wireless networks.